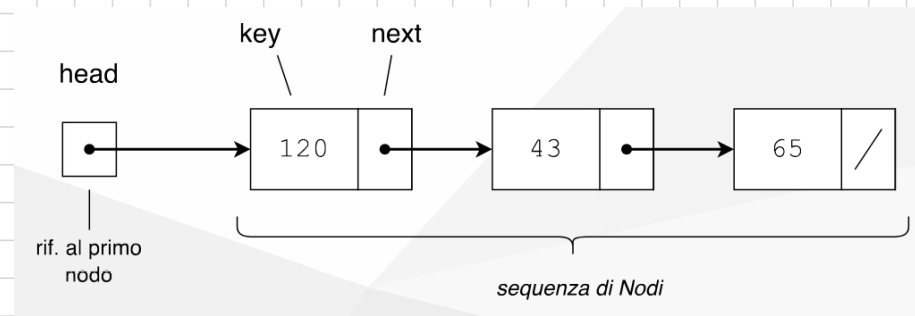


Linked list

Cos'è? : È una **struttura dati dinamica** formata da una **sequenza di nodi**, dove ogni nodo contiene:

- 1) Un **valore / dato**
- 2) Un **puntatore / riferimento** al **nodo successivo** nella sequenza (e a volte anche a quello precedente nelle versioni più avanzate)

A differenza degli array, in una linked list gli elementi **non sono memorizzati in posizioni contigue di memoria**. Invece, ogni nodo "punta" al **nodo successivo** (e anche al precedente nelle liste a doppia concatenazione)



Questa è una linked list semplice (singola concatenazione)

Caratteristiche principali:

- 1) **Dinamicità**: La dimensione della lista può crescere / ridursi durante l'esecuzione

- 2) **Non indicizzata**: Non si può accedere direttamente ad un elemento, oleri attraversare i nodi 1 a 1

- 3) **Facile inserimento / rimozione**: Ottimizzata per aggiunte o cancellazioni in mezzo alla lista senza dover spostare altri elementi

- 4) **Accesso sequenziale**: Per trovare un elemento oleri partire dal primo nodo

Liste semplici:

Struttura dati di base: Definiamo una classe (inner class) **Node** per rappresentare ciascun nodo e manteniamo un riferimento **head** al primo nodo

```
public class SimpleList {
```

```
    public static class Node {
```

```
        int key; // Sarebbe il valore/dato da salvare, può essere qualsiasi cosa
        Node next; // Riferimento che punta al prossimo nodo
```

```
    public Node (int key) {
```

```
        this.key = key; // Un nodo nasce con un valore/dato ma senza un riferimento al prossimo nodo
```

```
        this.next = null;
```

```
    }
```

```
    private Node head;
```

```
}
```

is Empty?: Una lista vuota non ha nodi, quindi **head = null**

Di una lista concatenata in genere non si lavora con la sua dimensione, ma si vuole sapere solo se è vuota o se ha almeno un elemento

```
boolean is Empty() {
```

```
    return this.head == null;
```

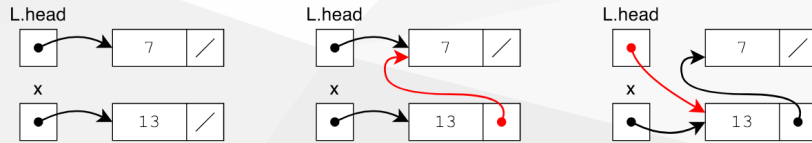
```
}
```

Inserimento in testa - Metto il nuovo elemento in cima

Inserimento con lista vuota:



Inserimento con lista non vuota:



1) Crea un nuovo nodo con il valore che vuoi inserire

2) Punta il campo successivo di questo nuovo nodo all'attuale testa della lista

3) Aggiorna testa per farla puntare al nuovo nodo

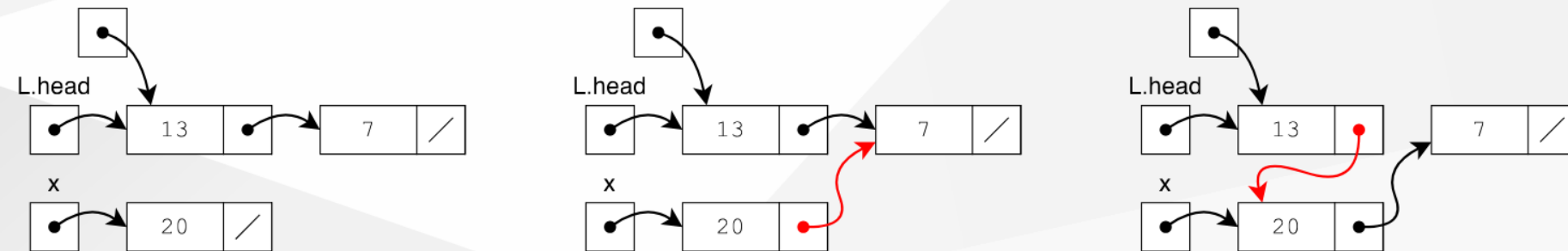
```
public Node prepend(Node x) {
```

```
    x.next = this.head; // la mia vecchia testa diventa la coda di x
```

```
    this.head = x; // il nuovo nodo viene messo in testa
```

Inserimento dopo un nodo dato

`insertAfter(Node i, Node x)` inserisce il nuovo nodo `x` come successore del nodo `i` già presente in lista. Se `i == null` allora è inserimento in testa

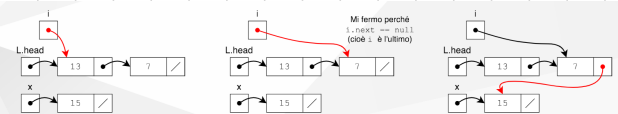


```
public Node insertAfter(Node i, Node x) {
    if (i == null) {
        prepend(x); // Inserimento in testa
    }
    else { // inserimento di x dopo i
        x.next = i.next;
        i.next = x;
    }
    return x;
}
```

Tempi: $O(1)$ (Sono tutte operazioni a costo costante)

Implemento in code

Per inserire in code x devo trovare il riferimento i all'ultimo elemento, poi procedo come da `insert After (i, x)`



Caso particolare quando la lista è vuota (= `prepend`):



```
public Node append (Node x) {
```

```
    if (this.head == null) {
```

```
        this.head = x
```

```
    } else {
```

```
        // Cerco l'ultimo nodo (cioè il nodo con next == null)
```

```
        Node i = this.head;
```

```
        while (i.next != null) { // dato che head != null, allora i.next non mi darà errore
```

```
            i = i.next
```

```
        } // Quando esco sarò arrivato all'ultimo nodo
```

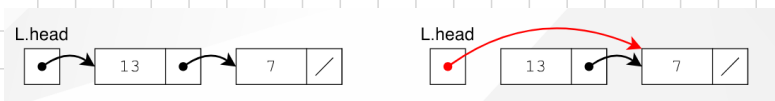
```
        i.next = x;
```

```
    }
```

```
    return x;
```

```
}
```

Concellazione in testa



Tempi = $O(1)$

```
public Node deleteFirst () {
```

```
    if (isEmpty) { // se la lista è vuota non faccio nulla
```

```
        return null
```

```
    }
```

```
    Node x = head
```

```
    head = head.next
```

```
    x.next = null;
```

```
    return x;
```

```
}
```

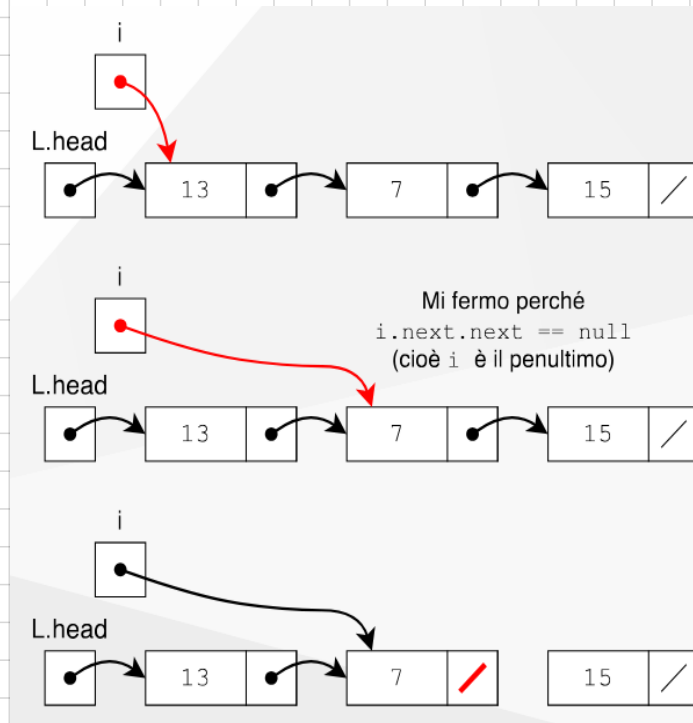
Concellazione in code

Come per l'operazione in code devo trovare la lista ma devo cercare il penultimo elemento (e poi assegnare null come successore)

- non esiste il penultimo
- cancellarlo equivale a svuotare la lista

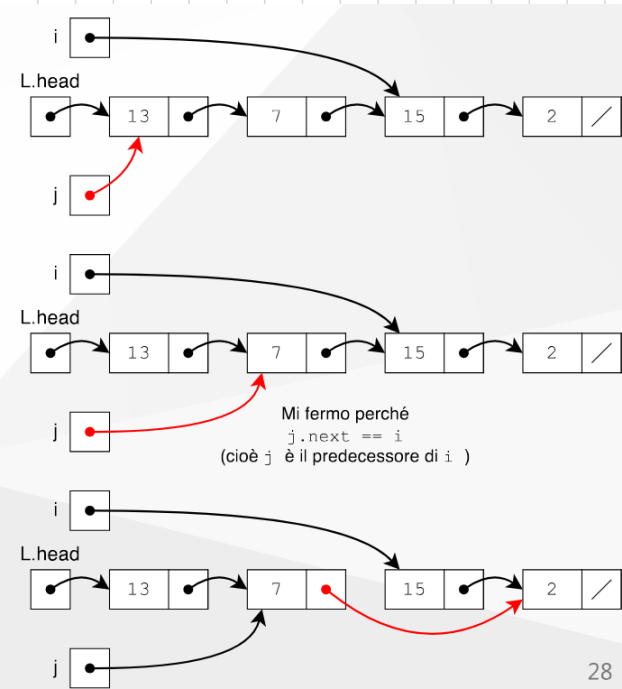
```
public Node deleteLast() {  
    // se la lista è vuota non faccio nulla  
    if (isEmpty()) {  
        return null;  
    }  
    // se head.next == null ho un solo elemento -> lo cancello  
    if (head.next == null) {  
        Node x = head;  
        head = null;  
        return x;  
    }  
    // Cerco il penultimo  
    while (i.next.next != null) {  
        i = i.next;  
    }  
    // arrivato al penultimo cancello il successore  
    Node x = i.next;  
    i.next = null;  
    return x;  
}
```

Tempi: $O(n)$



Cancellazione di un nodo dato

Per cancellare il nodo i devo cercare il suo predecessore j così da poter aggiornare il suo next a i .next



```
public Node delete (Node i) {  
    if (head == i) { // se head == i allora i non ha predecessore  
        head = i.next;  
    }  
    else {  
        Node j = head; // Cerco il predecessore  
        while (j.next != i) {  
            j = j.next;  
        }  
        // aggiorno il suo successore "saltando" i  
        j.next = j.next.next;  
    }  
    i.next = null;  
    return i;  
}
```

